

Diagnose R9700 Mapping Issues

08/23/2026 22:24:31

<https://chatgpt.com/c/6a8b1b3a-b178-83ea-b486-083c67d02ab0>

I'll reconstruct the new triage sequence, separate observations from interpretations, then validate th...



I'll reconstruct the new triage sequence, separate observations from interpretations, then validate the highest-impact claims against the relevant GFX12, tinygrad, and Linux paths.

The MQD address fix appears genuine, shifting the fault from CPF to a TCP shader write. I'm checking descriptor/ISA details before attributing that write to pointer or wave corruption.

The remaining evidence now points more specifically toward a bad workgroup-ID preload, so I'm validating the relevant GFX12 descriptor and dispatch fields.

Verdict

The triage found and fixed a **real MQD address-domain bug**. That work should stay.

However, the investigation then moved too quickly from the new TCP write fault to the MEC's `0xc672` exception. Before treating the RS64 exception as the remaining root cause, I see:

1. **One definite dispatch-geometry bug:** the RMSNorm probe is launching 64 workgroups, not one.
2. **One high-confidence launch-mode mismatch to verify immediately:** the hard-coded dispatch initiator does not appear to set wave32 mode, while the compiled GFX12 image may require it.
3. **Strong arithmetic evidence that the bad shader address is derived from an invalid workgroup ID,** rather than simply being a randomly corrupted output pointer.

I would fix and isolate those three things before inspecting MEC firmware state or continuing the transcendental work.

What the triage proved correctly

The MQD fix is well-supported:

```
raw VRAM/BAR offset: 0x0000000002003000
MC address:          0x0000008002003000
```

Changing `CP_MQD_BASE_ADDR` from the raw offset to `mc_base + kMqdPaddr` matches tinygrad's distinction between the physical placement of the MQD and the MC address consumed by the command processor. The focused suite passed 24 tests, and the existing C0 add-one proof still completed successfully. `native-r9700-session-full-4.jsonl`

The fault also changed in a way consistent with that fix:

- The previous CPF read/mapping fault at `0x02003000` disappeared.
- `CP_MEC_RS64_EXCEPTION_STATUS` changed from `0xc67a` to `0xc672`, clearing the RS64 page-fault bit while retaining the misaligned-address bit.
- The new GFXHUB fault is a **TCP shader write** under VMID 0 to an unmapped or non-writable page.

The new fault status `0x00041157` decodes as:

```
MORE_FAULTS      = 1
WALKER_ERROR     = 3
PERMISSION_FAULTS = 5
MAPPING_ERROR    = 1
CID              = 8  (TCP)
RW               = 1  (write)
VMID             = 0
```

Linux documents TCP as a shader-side client and defines the permission bits as invalid PTE, missing read, missing write, and missing execute. ([GitHub](#))

The CPU-side kernarg construction also looks correct. The output pointer inserted at kernarg offset 16 matches the allocated normalized-output buffer:

```
hidden input = 0x0000200000224000
scale        = 0x0000200000225000
output       = 0x0000200007628000
epsilon      = 0x3727c5ac
```

That substantially lowers the probability of a simple host-side ABI offset error. It does not yet prove that the GPU loaded the same pointer, but the host representation is sound.

Definite problem: `DISPATCH_DIRECT` is receiving the wrong units

The native PM4 builder programs:

```
COMPUTE_NUM_THREAD_X = config.workgroup_x;
PACKET3_DISPATCH_DIRECT DIM_X = config.global_x;
```

`native-r9700-session-full-4.jsonl`

The RMSNorm stage is built with:

```
stage.workgroup_x = 64;
append_stage(0, ..., 64);
```

`native-r9700-session-full-4.jsonl`

The transcript repeatedly interprets that as “64 total threads, one workgroup.” For direct PM4, that interpretation is wrong.

Tinygrad programs the local workgroup size through `COMPUTE_START_X` and sends `global_size` directly as the three `DISPATCH_DIRECT` dimensions. When it constructs an AQL packet, it converts the same values to work-items by multiplying `global_size × local_size`. In other words, the direct-PM4 dimensions are **workgroup counts**, not total work-items. ([GitHub](#))

Your current stage-0 packet therefore means:

```
local size:      64 threads
workgroup count: 64
total threads:   4,096
```

But the probe kernel does:

```
row = __builtin_amdgcn_workgroup_id_x();
row_offset = row * 2048;

hidden_output[row_offset + column] = result;
```

and the normalized output buffer is only one row-2,048 fp16 values, or 4,096 bytes. `native-r9700-session-full-4.jsonl`

Consequently:

- workgroup 0 writes the intended row;

- workgroup 1 writes one page beyond it;
- workgroups 2-63 continue writing subsequent pages.

That is enough by itself to invalidate the current epsilon test and generate multiple TCP write faults. The reported `MORE_FAULTS=1` is consistent with more than one bad store being observed.

For this kernel the launch must be:

```
local_x      = 64
group_count_x = 1
```

Do not blindly divide every stage's current `global_x` by 64. Audit each kernel's indexing model. Some may intentionally assign one workgroup per output row, head, or tile. But stage 0 is unambiguous: it produces one 2,048-element row and therefore requires one workgroup.

I would also rename the fields:

```
global_x -> group_count_x
global_y -> group_count_y
global_z -> group_count_z
```

That removes a dangerous ambiguity between HSA grid sizes in work-items and PM4 dispatch dimensions in workgroups.

Likely second problem: wave32 is not being set

The native dispatch initiator is currently:

```
constexpr uint32_t encode_dispatch_initiator() {
    return (1U << 0) | (1U << 2);
}
```

which produces `0x00000005`. `native-r9700-session-full-4.jsonl`

Those bits represent the compute-shader enable and force-start-at-zero controls. They do not include the canonical `CS_W32_EN` bit at bit 15. LLVM explicitly identifies bit 15 of `COMPUTE_DISPATCH_INITIATOR` as `CS_W32_EN`. ([Debian Sources](#))

Tinygrad does not hard-code this value. It derives wave mode from the program descriptor and emits:

```
regCOMPUTE_DISPATCH_INITIATOR.encode(
    cs_w32_en=int(prg.wave32),
    force_start_at_000=1,
    compute_shader_en=1,
)
```

for modern AMD targets. ([GitHub](#))

Therefore, before the next hardware run:

1. Decode `ENABLE_WAVEFRONT_SIZE32` from the kernel descriptor.
2. Log the decoded wave mode.
3. Log the exact initiator value.
4. Set `CS_W32_EN` from that descriptor property.

LLVM documents the descriptor property and the distinction between wave64 and native wave32 execution. ([LLVM](#))

If the epsilon image is wave32, the expected initiator under the conventional layout would be:

```
0x00008005
```

rather than `0x00000005`.

I would not hard-code `0x8005` until the actual image's descriptor has been decoded. The durable fix is a target-derived field encoder equivalent to tinygrad's, not another literal.

This also offers a plausible explanation for why the C0 add-one proof passes while generated HSA images fail: the hand-authored C0 kernel may use a different wave mode and does not necessarily exercise the same system-SGPR contract.

The fault address strongly implicates `workgroup_id_x`

The correct output base is:

```
0x0000200007628000
```

The two captured fault pages correspond to:

```
0x000020006851f000  
0x0000200068530000
```

Subtracting the output base gives:

```
0x60ef7000 = 397,047 × 4,096  
0x60f08000 = 397,064 × 4,096
```

The difference between the two implied values is 17.

That is unusually specific because the kernel's per-row stride is exactly:

```
2,048 fp16 elements × 2 bytes = 4,096 bytes
```

So, assuming the GPU loaded the correct output base, the captured fault addresses correspond exactly to:

```
workgroup_id_x = 397,047  
workgroup_id_x = 397,064
```

This is an inference, not something directly printed by the GPU. But it is a much better fit than “the pointer was randomly corrupted,” especially because the value changes by a small integer number of rows between runs.

The descriptor's `RSRC2=0x84` is relevant:

```
USER_SGPR_COUNT = 2  
TGID_X_EN       = 1
```

LLVM documents bit 7 as enabling Workgroup ID X and states that the workgroup ID SGPR follows the user-data SGPRs in the initial kernel state. CP supplies user-data SGPRs, ADC supplies workgroup IDs, and SPI supplies work-item IDs. ([LLVM](#))

The expected initial SGPR layout is therefore approximately:

```
s0:s1 = kernarg pointer
s2    = workgroup_id_x
```

Your kernarg pointer path appears correct. The suspicious component is the system-SGPR/workgroup-ID launch state.

However, because the current packet launches 64 groups and the fault record says `MORE_FAULTS=1`, do not conclude that `s2` is corrupt from the current run alone. First reduce the launch to one workgroup and establish a clean fault baseline.

Recommended next experiment sequence

1. Fix only the stage-0 group count

Change stage 0 to:

```
workgroup_x = 64;
group_count_x = 1;
```

Add a no-hardware PM4 test asserting:

```
COMPUTE_NUM_THREAD_X = 64
DISPATCH_DIRECT.DIM_X = 1
```

Do not change the arithmetic kernel yet.

2. Decode and program wave mode

Extend the image metadata with:

```
wave32
enable_wavefront_size32
kernel_code_properties
```

Then derive the dispatch initiator from it. Log:

```
RSRC1
RSRC2
RSRC3
wave32
dispatch_initiator
user_sgpr_count
tgid_x_en
workgroup size
group count
```

This should become a launch-contract assertion, not merely debug output.

3. Use three tiny A/B kernels through the same resident path

Run exactly one workgroup for each.

A. Pointer-only constant store

```
if (__builtin_amdgcn_workitem_id_x() == 0) {
    reinterpret_cast<volatile unsigned int*>(hidden_output)[0] = 0x12345678;
}
```

No workgroup ID and no floating-point operation.

- If this faults, the remaining problem is pointer loading, kernarg visibility, wave mode, or broader launch state.
- If it passes, the GPU-loaded output pointer and basic store path are good.

B. Workgroup-ID sentinel

```
if (__builtin_amdgcn_workitem_id_x() == 0) {
    auto* out = reinterpret_cast<volatile unsigned int*>(hidden_output);
    out[0] = __builtin_amdgcn_workgroup_id_x();
    out[1] = __builtin_amdgcn_workitem_id_x();
}
```

Use fixed output indices-do not index the destination by `row`.

With one group, the expected result is:

```
out[0] = 0
out[1] = 0
```

- If A passes and B reports a large or unstable `out[0]`, you have isolated the TGID/system-SGPR setup.
- If both pass, the launch-state basics are sound.

C. Epsilon arithmetic with hard-coded row zero

Only after A and B pass:

```
const unsigned int row = 0;
```

Keep the epsilon operation otherwise unchanged.

- If C passes, the problem was workgroup-ID or dispatch geometry.
- If C fails, then the transcendental lowering becomes relevant again.

Clean-state requirements

Before each candidate:

1. Fully dequeue/reset the HQD.
2. Verify `CP_HQD_ACTIVE == 0`.
3. Record the GCVN and RS64 fault registers.
4. Clear them using a source-grounded sequence-or cold-reset if no reliable clear sequence is known.
5. Verify the baseline is clear.
6. Run one true resident constant-store sentinel.
7. Run one candidate.

8. Read the sentinel again after the candidate.

Do not use the separate C0 add-one path as the only sentinel; it does not necessarily exercise the generated-image wave mode, system SGPRs, and stage PM4 configuration.

The transcript also still shows a live `CP_HQD_PQ_CONTROL` value differing from the intended descriptor by bit 28. Treat that as another indication that queue state should be asserted before every diagnostic run rather than assumed.

What I would not investigate yet

I would pause these lines of inquiry:

- MEC firmware instruction `0xc67` ;
- one-op sqrt/reciprocal/rsqrt microkernels;
- further LLVM division-lowering analysis;
- more cache-invalidation variations;
- further RSRC3 overrides.

`0xc672` tells you the RS64 observed a misaligned-address exception with the earlier page-fault bit cleared. It does not prove that the MEC corrupted a shader pointer or workgroup ID. The transcript's final theory—"the MEC aborts launch and leaves garbage wave state"—is possible, but currently speculative.

You now have two much more direct, testable launch defects:

1. **stage 0 is dispatched as 64 workgroups instead of one;**
2. **the wave32 launch bit is not derived from the image descriptor.**

Correct those first. My strongest expectation is that the combination of one-group geometry and descriptor-derived wave mode will either make the epsilon probe retire or reduce the remaining fault to a clean, interpretable A/B result.

Sources

<https://github.com/torvalds/linux/blob/master/Documentation/gpu/amdgpu/debugging.rst> github.com

https://raw.githubusercontent.com/tinygrad/tinygrad/master/tinygrad/runtime/ops_amd.py raw.githubusercontent.com

https://sources.debian.org/src/llvm-toolchain-11/1%3A11.0.1-2/llvm/test/CodeGen/AMDGPU/mixed_wave32_wave64.ll sources.debian.org

<https://llvm.org/docs/AMDGPUUsage.html> llvm.org